Ecosystem User Manual

Network Automation Tool Integration Guide

Simon Knight

Adelaide, Australia

March 2026

> **About this manual.** This manual guides practitioners through end-to-end workflows that compose multiple tools: topology generation, modelling, simulation, visualization, deployment, and validation. It covers the five primary integration workflows, the data contracts (RFC-01 and RFC-02) that bind the tools together, tool selection guidance, and troubleshooting for known integration gaps. Readers who need depth on any individual tool should consult its dedicated user manual, cross-referenced throughout.

Contents

1 Quick Start

The fastest way to experience the ecosystem is the **Rapid Prototyping** workflow: generate a topology, simulate it, visualize it. Three commands and under two minutes.

Step 1. Install the core trio of tools:

```
cargo install topogen
cargo install netsim
pip install netvis
```

Step 2. Generate a seed-deterministic Fat-Tree topology and export it directly to netsim format:

```
topogen generate --template fat-tree --k 4 --seed 42 \
    --format netsim --output lab.topo.yaml
```

This produces `lab.topo.yaml` with 20 nodes, logical interface IDs (`p1`, `p2`, …), and an OSPF-ready adjacency map.

Step 3. Simulate protocol convergence:

```
netsim run lab.topo.yaml --output lab.results.json
```

netsim runs a deterministic OSPF convergence simulation and writes the FIB state, convergence times, and reachability matrix to `lab.results.json`.

Step 4. Visualize the topology with simulation overlays:

```
netvis render lab.topo.yaml --overlay lab.results.json \
    --output lab.svg
```

Open `lab.svg` in any browser. Nodes are colored by convergence status; links show utilization from the FIB walk.

All three tools accept `--seed` for reproducible output. The same seed produces identical topology, simulation results, and layout across runs—enabling CI/CD regression testing.

2 Installation & Prerequisites

2.1 System Requirements

▷ **Prerequisites**

- **Python 3.10 or later** — required by autonetkit, netvis (PyO3 bindings), and config-parsing
- **Rust 1.70 or later** — required by topogen, netsim, netflowsim, NTE, and netvis native binary
- **Node.js 18+** — required by Workbench (React frontend)
- **pip or uv** — Python package manager (`uv` recommended for faster dependency resolution)
- **Git** — required by RFC-02 project manifest `git` provider
- 4 GB RAM minimum; 8 GB recommended for NTE graph workloads exceeding 10K nodes

2.2 Installing the Tools

The ten tools span two languages and three distribution channels. Install only the tools you need for your workflow (see Section 5 for guidance).

Rust tools (via Cargo):

```
cargo install topogen        # topology generation
cargo install netsim         # protocol simulation
cargo install netflowsim     # traffic flow analysis
cargo install ank-netcfg     # Rust-native config compiler
```

Python tools (via pip or uv):

```
pip install autonetkit       # network modeling library
pip install netvis           # visualization (PyO3 + WASM)
pip install netassure        # advanced analysis (early preview)
pip install configparsing    # LLM/RAG config extraction
```

NTE (Network Topology Engine, Python bindings to Rust core):

```
pip install ank-nte
```

Workbench (full web IDE):

```
pip install ank-workbench
ank-workbench serve          # starts on http://localhost:8080
```

Verify installs:

```
topogen --version   # expect: topogen 1.5.x
netsim --version    # expect: netsim 2.1.x
ank --version       # expect: autonetkit 1.8.x
netvis --version    # expect: netvis 1.9.x
```

> **Tip**
>
> You do not need to install every tool upfront. Start with the tools for your immediate workflow and add others incrementally. The project manifest (`netauto.project`, Section 4.3) will indicate which tools are needed as you extend your pipeline.

> **Caution**
>
> The tools span a wide maturity range. **netsim** (v2.1) and **netvis** (v1.9) are production-ready with comprehensive test suites (2,616 and 1,535 tests respectively). **netassure** (v0.1) is in early architecture phase—its five analysis paradigms are designed but not yet implemented. Do not include netassure in production pipelines without explicit validation.

2.3 Verifying Cross-Tool Connectivity

Run a quick smoke test to confirm the core integration path works on your system:

```
topogen generate --template ring --nodes 6 --seed 1 \
    --format netsim --output /tmp/smoke.topo.yaml
netsim run /tmp/smoke.topo.yaml --output /tmp/smoke.results.json
netvis render /tmp/smoke.topo.yaml \
    --overlay /tmp/smoke.results.json --output /tmp/smoke.svg
echo "Smoke test passed"
```

3 End-to-End Workflows

The ecosystem supports five distinct end-to-end workflows. Each workflow describes which tools participate, what data passes between them, and any known integration gaps.

3.1 Workflow 1: Rapid Prototyping

Use case: "Generate a topology, simulate it, see if it works." The fast inner loop for validating network designs before investing in a container lab.

Tools: topogen → netsim → netvis

Status: Fully operational. topogen exports directly to netsim format. All three tools support the `--seed` flag for reproducible output.

Step 1. Generate the topology. Choose a template that matches your design intent:

```
# Data centre: Fat-Tree with k=4 (20 nodes)
topogen generate --template fat-tree --k 4 --seed 42 \
    --format netsim --output dc.topo.yaml

# WAN: 8-node ring with OSPF-tagged overlays
topogen generate --template ring --nodes 8 --seed 42 \
    --overlays ospf --format netsim --output wan.topo.yaml

# Large-scale: Barabasi-Albert random graph (128 nodes)
topogen generate --template barabasi-albert \
    --nodes 128 --m 3 --seed 42 \
    --format netsim --output scale.topo.yaml
```

Available templates: `fat-tree`, `leaf-spine`, `ring`, `mesh`, `hierarchical`, `pop`, `barabasi-albert`, `watts-strogatz`, `erdos-renyi`.

Step 2. Inspect the generated topology:

```
topogen show dc.topo.yaml --summary
```

The summary reports node count, link count, interface naming (logical IDs `p1`, `p2`, …), and any detected structural issues.

Step 3. Run the protocol simulation:

```
netsim run dc.topo.yaml --output dc.results.json
```

netsim performs deterministic OSPF convergence simulation. Output includes: converged FIB per node, convergence time per link failure scenario, and a reachability matrix. Expect under 10 seconds for topologies up to 200 nodes.

Step 4. Check for convergence issues:

```
netsim report dc.results.json --check convergence
```

Any non-convergent paths or routing loops are reported with the affected node pair and FIB snapshot at failure time.

Step 5. Visualize with simulation overlay:

```
netvis render dc.topo.yaml --overlay dc.results.json \
    --layout force --output dc.svg
```

```
```

The SVG output colors nodes by convergence status (green = converged, amber = degraded, red = unreachable) and annotates links with FIB-derived utilization percentages.

Step 6. (Optional) Export to interactive HTML for browser exploration:

```
netvis render dc.topo.yaml --overlay dc.results.json \
    --format html --output dc.html
```

See also: *TopoGen User Manual* (TOPOGEN-UM-001) — Generator templates, traffic matrix configuration, and overlay tagging.

See also: *netsim User Manual* (NETSIM-UM-001) — Protocol configuration, failure scenarios, and FIB inspection.

See also: *netvis User Manual* (NETVIS-UM-001) — Layout algorithms, overlay types, and export formats.

3.2 Workflow 2: Design-to-Deploy

Use case: "Model a network declaratively, generate vendor-specific configurations, validate with simulation, then deploy to a lab or production environment."

Tools: autonetkit → netsim (validation) → autonetkit-config / ank-netcfg → ContainerLab

Status: Operational. One known gap: simulation results do not yet propagate back to the design layer (see Section 7).

Step 1. Create a topology file. You can use topogen (Workflow 1, Step 1) or write one manually:

```
# Or generate from topogen and convert to design format:
topogen generate --template leaf-spine --spines 2 --leafs 4 \
    --seed 10 --output spine.topo.yaml
```

Step 2. Create a design sidecar with protocol intent. The design file references the topology via `base_topology` and uses logical interface IDs (`p1`, `p2`):

```
ank init --topology spine.topo.yaml \
    --output spine.design.yaml \
    --protocols ospf,bgp
```

Edit `spine.design.yaml` to set OSPF areas, BGP AS numbers, and IP address plans. The design file references only logical IDs; vendor-specific names are resolved at compile time.

Step 3. Validate the design against autonetkit's 31 built-in rules:

```
ank validate spine.design.yaml --strict
```

Validation covers graph structure, resilience, addressing, cross-endpoint consistency, routing protocol correctness, and hierarchy. Fix any reported errors before proceeding.

Step 4. Pre-validate with netsim simulation:

```
ank export spine.design.yaml --format netsim \
    --output spine.netsim.yaml
netsim run spine.netsim.yaml --output spine.results.json
netsim report spine.results.json --check convergence,loops
```

This step catches routing issues before any hardware is touched.

Step 5. Compile vendor configurations. Choose between the Python library (autonetkit-config) and the Rust CLI (ank-netcfg):

```
# Python path (interactive / library use):
ank build spine.design.yaml --platform junos \
    --output configs/

# Rust CLI path (CI/CD / deterministic pipelines):
ank-netcfg compile spine.design.yaml \
    --platform junos --output configs/
```

Supported platforms: `ios`, `junos`, `eos`, `ios-xr`, `nxos`, `vrp`, `srlinux`, `sonic`, and three others. See NETCFG-UM-001 for full list.

Step 6. Export a ContainerLab topology and deploy:

```
ank export spine.design.yaml --format containerlab \
    --output clab-spine.yaml
sudo containerlab deploy --topo clab-spine.yaml
```

Step 7. (Optional) Validate the deployed network with deviceinteraction:

```
di run --testbed clab-spine.yaml \
    --baseline spine.results.json \
    --suite routing
```

deviceinteraction connects to live devices via SSH, runs the routing verification suite, and compares results against the netsim baseline.

> **Tip**
>
> Use `ank-netcfg` for CI/CD pipelines where determinism and auditability are priorities: it produces identical output for identical inputs and emits a trace log for every configuration decision. Use `ank build` for interactive Python workflows where you need library access to the topology object model.

See also: *AutoNetKit User Manual* (ANK-UM-001) — Design file schema, validation rules, and the query API.

See also: *NetCfg User Manual* (NETCFG-UM-001) — Compilation targets, template customization, and vendor platform support.

3.3 Workflow 3: Traffic Engineering

Use case: "Analyze capacity and performance. What happens to utilization if traffic doubles? Where are the bottlenecks?"

Tools: topogen → netflowsim (capacity planning); netsim → netflowsim (advanced, via FIB bridge in Workbench)

Status: The topogen → netflowsim path is operational. The netsim → netflowsim CLI bridge is not yet available; use Workbench (Section 3.4) for the combined protocol + traffic workflow.

Step 1. Generate a topology with a traffic matrix. topogen's gravity model produces realistic demand proportional to node weight and inter-node distance:

```
topogen generate --template hierarchical \
    --pops 3 --nodes-per-pop 8 --seed 77 \
```

```
    --traffic-matrix gravity \
    --traffic-asymmetry 0.2 \
    --output wan.topo.yaml \
    --traffic-output wan.traffic.csv
```

`wan.traffic.csv` contains source/destination/demand tuples in Gbps.

Step 2. Run the flow simulation:

```
netflowsim run wan.topo.yaml \
    --traffic wan.traffic.csv \
    --iterations 10000 \
    --output wan.flowresults.json
```

netflowsim uses a Monte Carlo engine (per-thread seeded RNG, parallelized with Rayon) to compute expected link utilization, 99th-percentile queue depths, and flow completion times across the demand matrix.

Step 3. Identify bottlenecks:

```
netflowsim report wan.flowresults.json --top-links 10
```

Reports the ten highest-utilization links with mean and 99th-percentile utilization.

Step 4. Visualize traffic heat map:

```
netvis render wan.topo.yaml \
    --overlay wan.flowresults.json \
    --overlay-type traffic-heatmap \
    --output wan-traffic.svg
```

Links are colored on a green-to-red gradient by utilization.

Step 5. (Optional) Model a capacity failure scenario—remove a link and re-run to see utilization shift:

```
netflowsim run wan.topo.yaml \
    --traffic wan.traffic.csv \
    --fail-link pop1-core:p1--pop2-core:p1 \
    --output wan.failure.json
```

See also: *Spectral/netflowsim User Manual* (SPECTRA-UM-001) — Queuing models, Monte Carlo configuration, and failure scenario syntax.

3.4 Workflow 4: Interactive Exploration (Workbench)

Use case: "I want a single web interface for design, editing, simulation, and visualization—without switching between CLI tools."

Tools: Workbench (orchestrates netsim and netvis as subprocesses via gRPC and JSON)

Status: Operational at v4.2. The Workbench provides the only available CLI-level FIB bridge from netsim to netflowsim.

Step 1. Start the Workbench server:

```
ank-workbench serve --port 8080
```

Open `http://localhost:8080` in a browser. The Workbench requires netsim and netvis to be installed and on `$PATH`.

Step 2. Create or import a project. Use the **New Project** wizard to:

- Enter a project name (creates `netauto.project`)
- Select a topology source: *blank*, *topogen template*, or *import YAML*
- Choose initial protocols: OSPF, BGP, or both

Step 3. Edit the topology in the YAML editor (left panel). The Workbench validates YAML structure against 27 rules as you type, highlighting errors inline.

Step 4. Run simulation. Click **Simulate** (or press `Ctrl+R`) to invoke netsim as a background subprocess. The status bar shows simulation progress; results appear in the right panel when complete.

Step 5. Explore the visual topology (center panel). The netvis WASM renderer displays the topology with simulation overlays. Pan and zoom with mouse; click a node to see its FIB table and interface list.

Step 6. (Advanced) Enable the FIB bridge to run traffic analysis. From the **Analysis** menu, select **Traffic Engineering**. The Workbench converts the netsim FIB output to netflowsim format internally and launches a flow simulation run.

Step 7. Open an interactive terminal session to a simulated node (netsim daemon mode, gRPC):

```
# From within the Workbench Terminal tab:
netsim> show ip route vrf default
netsim> ping 10.0.0.5 source 10.0.0.1
```

> **Tip**
>
> The Workbench is the recommended entry point for users new to the ecosystem. It hides the file-format details of RFC-01 while still producing standard `.topo.yaml`, `.design.yaml`, and `.results.json` sidecars that other CLI tools can consume.

See also: *Workbench User Manual* (ANK-WB-UM-001) — Project management, editor configuration, simulation options, and terminal usage.

3.5 Workflow 5: Closed-Loop Assurance (Future)

Use case: "Design, deploy, monitor operational state, predict failures, and automatically remediate drift from intent."

Tools: autonetkit → netsim → deploy → deviceinteraction → netassure → (feedback) → autonetkit

Status: Planned. This workflow requires netassure (v0.1, architecture phase only) and the deviceinteraction CLI (Phase 8, not yet shipped). It is documented here as the long-term integration vision.

The Closed-Loop Assurance workflow extends Workflow 2 with three additional stages:

1. **Operational capture.** After deployment, deviceinteraction connects to live devices via SSH/Telnet/REST, executes test triggers, parses CLI output, and produces an `.operational.json` snapshot aligned to RFC-01 identity conventions. configparsing can augment this with LLM-extracted topology relationships from raw vendor configurations.
2. **Assurance analysis.** netassure consumes three data sources simultaneously: the static design (`.topo.yaml`, `.design.yaml`), simulation results (`.results.json`), and runtime telemetry (Prometheus, BMP, NetFlow, gNMI). Its five analysis paradigms (formal verifica-

tion, graph algorithms, failure cascade modeling, ML/GNN prediction, optimization) detect drift, predict failures, and generate ranked remediation recommendations.

3. **Automated remediation.** Remediation recommendations are fed back into autonetkit as design annotations. For drift (e.g., an interface is up in the design but down in operational state), a remediation workflow re-runs the relevant portion of Workflow 2 automatically.

When fully implemented, this loop enables "intent-based networking" at the ecosystem level: the declared design is continuously reconciled against operational reality, with human oversight at configurable escalation thresholds.

See also: *Passive/netassure User Manual* (PR-UM-001) — Analysis paradigms, telemetry ingestion, and GNN inference.

See also: *NTE User Manual* (NTE-UM-001) — Graph-relational queries used internally by netassure.

4 Data Contracts

The tools interoperate through **versioned file-based contracts**, not through tight in-process coupling. Understanding these contracts is essential for debugging integration issues and extending the pipeline with custom tools.

4.1 RFC-01: The Multi-File Sidecar Strategy

RFC-01 defines a layered, file-based approach where each file has a single owner and a specific role. The key design principle is **logical-to-physical decoupling**: protocol designs reference abstract interface IDs (`p1`, `p2`), never vendor-specific names (`GigabitEthernet0/1`). If hardware changes from Cisco to Juniper, only the topology file's vendor mapping changes; the design file remains untouched.

The four RFC-01 artifacts:

`*.topo.yaml` — Physical World Owner: topogen or manual entry. Defines nodes, physical links, the logical-to-physical interface mapping table (abstract ID → vendor name), geographic coordinates, and hardware metadata. Example:

```
nodes:
  - id: nyc-spine-01
    vendor: cisco
    interfaces:
      - id: p1
        vendor_name: GigabitEthernet0/1
        speed: 100G
      - id: p2
        vendor_name: GigabitEthernet0/2
        speed: 100G
```

`*.design.yaml` — Logical Intent Owner: autonetkit-config or manual entry. Defines IP addressing, protocol configurations (OSPF, BGP, IS-IS), VRFs, and ACLs. References the topology via `base_topology` and uses only logical IDs:

```
schema: netauto/design/v2.0
base_topology: ./nyc.topo.yaml
protocols:
  ospf:
    area: 0
    interfaces:
      - node: nyc-spine-01:p1   # logical ID
```

```
        cost: 10
```

`*.results.json` — Execution Output Owner: netsim or netflowsim. Contains converged routing tables, link utilization, convergence timing, and error logs. References only Node and Interface IDs from the Topology/Design files—never invents new IDs.

`*.operational.json` — Operational State Owner: Normalizers (autonetkit-analysis, config-parsing, deviceinteraction). A versioned snapshot of live device state, aligned to RFC-01 identity conventions. Contract ID: `netauto/operational-topology/v1.0`.

4.2 ID Conventions

Stable, human-predictable identifiers are the linchpin of the sidecar strategy. If re-running topogen with the same seed produces different IDs, every downstream sidecar is invalidated.

Node ID Pattern: `[site]-[role]-[index]`. Examples: `nyc-spine-01`, `lon-leaf-03`. Deterministic from seed; never auto-incremented.

Interface ID (Logical) Pattern: `p1`, `p2`, … Abstract; decouples design from hardware. Used in `.design.yaml` and `.results.json`.

Link ID (Derived) Pattern: `node_a:pX--node_b:pY`. Never manually named; derived from endpoints. Example: `nyc-spine-01:p1--nyc-leaf-01:p1`.

> **Tip**
>
> Run `topogen show <file> –ids` to list all assigned IDs in a topology file. Use this output to verify ID stability when regenerating a topology with the same seed.

4.3 RFC-02: The Project Manifest

While RFC-01 defines the data artifacts, RFC-02 defines how artifacts are **associated and orchestrated**. The project manifest (`netauto.project`) is a lightweight YAML file—a "Makefile" for network engineering—that links sidecars together, tracks data sources, and coordinates tool execution.

```
project_name: "NYC-Spine-Design"
version: "1.0"

layers:
  topology:
    provider: "file"
    path: "./nyc.topo.yaml"
  design:
    provider: "git"
    url: "https://github.com/org/net-designs.git"
    ref: "main"
    path: "sites/nyc/ospf.design.yaml"

artifacts:
  simulation_results: "./results/nyc.results.json"
  flow_analysis:      "./results/nyc.flowresults.json"

hooks:
  telemetry:
    type: "websocket"
    url: "ws://telemetry.local:8080/stream/nyc"
    target: "netvis"
```

Supported providers:

RFC-02 Providers

Command	Description
`file`	Local file path; used for development and rapid prototyping
`git`	Git-hosted design; auto-fetched on `ank run`
`http`	HTTP/HTTPS endpoint; used for traffic matrices from monitoring APIs
`netbox`	Direct NetBox/Nautobot Source of Truth integration
`live`	Real-time network state; requires telemetry hooks

The Workbench, CLI (`ank`), and TUI (`ank-tui`) all consume `netauto.project`. You can switch between them at any point in a project lifecycle without data loss.

5 Tool Selection Guide

5.1 Start Here: Choose by Use Case

Use Case to Tool Mapping

Command	Description
`Generate a random or structured topology`	topogen
`Model a design with typed Python`	autonetkit
`Simulate OSPF/BGP protocol convergence`	netsim
`Analyze traffic capacity and bottlenecks`	netflowsim
`Visualize a topology (CLI)`	netvis
`Visualize and simulate interactively (web)`	Workbench
`Compile vendor device configurations`	ank-netcfg or autonetkit-config
`Parse legacy vendor configurations`	configparsing
`Validate live device state against design`	deviceinteraction
`Graph-relational topology queries (Python API)`	NTE via autonetkit
`Advanced analysis / ML prediction (future)`	netassure

5.2 Which Config Compiler?

Two tools compile network designs to vendor configurations:

autonetkit-config (Python) Best for: interactive Python workflows, library consumers, data science integration, and use cases needing direct access to the topology object model.

```
ank build design.yaml --platform ios --output configs/
```

ank-netcfg (Rust) Best for: CI/CD pipelines, auditable deployments, and scenarios where a single deterministic binary is preferred. Produces a trace log for every configuration decision.

```
ank-netcfg compile design.yaml --platform ios --output configs/
```

5.3 Maturity-Aware Tool Selection

Tool Maturity Summary

Command	Description
`netsim v2.1`	Production-ready; 2,616 tests; enterprise protocols
`netvis v1.9`	Production-ready; 1,535 tests; scale milestone shipped
`NTE v0.2`	Stable; 16-crate workspace; dual-write hardened
`topogen v1.5`	Stable; 869 tests; intent-based overlays
`Workbench v4.2`	Stable; web IDE with simulation and visualization
`autonetkit v1.8`	Active; batteries-included module
`ank-netcfg v1.2`	Active; front and back ends shipped
`configparsing v1.0`	Mid-maturity; v2.0 production layer in progress
`netflowsim v0.1`	Early; profiling phase; avoid for production
`netassure v0.1`	Early (architecture only); do not use in pipelines

6 Integration Architecture

6.1 File-Based Integration (Primary)

The primary integration mechanism between all tools is **file exchange** via RFC-01 sidecar files. Each tool reads its inputs from disk, produces its outputs to disk, and never mutates another tool's output file. This design preserves modularity, enables independent evolution, and makes every integration point inspectable with standard text tools (`cat`, `jq`, `diff`).

The natural data flow through the pipeline is:

1. **Generate** (`topogen`) → `.topo.yaml`
2. **Model** (`autonetkit` + NTE) → `.design.yaml`
3. **Simulate** (`netsim`, `netflowsim`) → `.results.json`
4. **Visualize** (`netvis`) → SVG/HTML/PNG
5. **Assure** (`netassure`, future) → analysis reports and predictions

The pipeline is not strictly linear. Tools can be entered at any stage: import from NetBox at Step 2, skip Step 3 and go directly to visualization, or loop back from Step 4 to Step 2 after discovering issues.

6.2 In-Process FFI: autonetkit ↔ NTE

Where performance demands it, autonetkit uses the Network Topology Engine (NTE) **in-process** via PyO3 (Python-to-Rust FFI). NTE is not a separate process; it runs inside the autonetkit Python process. This gives autonetkit sub-millisecond graph query latency without a network round-trip.

When you call `topology.query(...)` in autonetkit, you are invoking NTE's Cypher-inspired lazy query engine compiled to native Rust. The `TopologyInterface` ABC allows the same Python code to switch between `InProcessTopology` (NTE via FFI) and `RemoteTopology` (NTE server via HTTP/WebSocket) transparently.

6.3 Subprocess + Messaging: Workbench → netsim / netvis

The Workbench orchestrates netsim and netvis as **managed subprocesses**, not as importable libraries:

- **netsim daemon:** The Workbench launches netsim in daemon mode and communicates via gRPC over a Unix Domain Socket (UDS). This enables the interactive terminal experience (the simulated CLI) and real-time convergence events.
- **netvis subprocess:** The Workbench invokes netvis as a one-shot subprocess (JSON topology in, SVG/HTML out) and renders the result in the center panel.

This design allows netsim and netvis to evolve their APIs independently; the Workbench communicates only through the CLI contract.

6.4 CLI Pipeline: the `ank` Command

The `ank` command is the orchestration entry point for power users and CI/CD pipelines. It reads `netauto.project` and sequences tool invocations:

```
# Build the full pipeline from a project manifest:
ank run netauto.project --stages generate,simulate,visualize

# Run only validation and simulation:
ank run netauto.project --stages validate,simulate

# Diff two topology files:
ank diff before.topo.yaml after.topo.yaml
```

ank Commands

Command	Description
`ank run <project>`	Execute pipeline stages from project manifest
`ank validate <file>`	Run design validation (31 rules)
`ank build <file>`	Build vendor configs from design
`ank export <file>`	Export to netsim, containerlab, or json format
`ank diff <a> <b>`	Compare two topology or design files
`ank show <file>`	Display topology summary
`ank init`	Scaffold a new project with manifest
`ank version`	Print version and exit

7 Troubleshooting

Problem 1: Simulation results show non-convergent paths after design validation passed

Cause: autonetkit's 31 design-time rules catch structural and addressing errors, but protocol-level convergence issues (e.g., OSPF metric loops from misconfigured costs) are not detectable without simulation. Design validation and protocol simulation check different properties.

Solution: Run `netsim report <results.json> –check convergence,loops` to identify the affected node pairs and FIB state. Inspect the `.design.yaml` OSPF costs for the flagged links. There is currently no automated feedback mechanism from simulation to design; you must manually diagnose and update the design file.

Problem 2: netflowsim: FIB bridge not available from the CLI

Cause: The netsim → netflowsim FIB bridge (which converts converged FIB state into netflowsim's flow graph format) is only available inside the Workbench v4.2 orchestration

layer. A standalone CLI tool for FIB export (`netsim export –format netflowsim`) is planned but not yet shipped.

Solution: Use the Workbench for combined protocol + traffic analysis workflows (Workflow 4, Section 3.4). Select **Analysis → Traffic Engineering** from the Workbench menu to invoke the bridge automatically. For CLI-only pipelines, file a feature request referencing the FIB bridge gap.

Problem 3: Workbench does not detect design anti-patterns (protocol errors, SPOFs)

Cause: The Workbench validates YAML structure against 27 syntactic rules but treats netsim and netvis as black boxes. It cannot reason about topology semantics: OSPF area mis-configuration, single points of failure, or BGP session asymmetry are not visible to the Workbench directly.

Solution: For semantic validation, run `ank validate <design.yaml> –strict` from the CLI (31 rules covering protocol correctness and resilience), then run netsim simulation to check convergence. The long-term target is Workbench integration with NTE or autonetkit as an embedded library for topology-aware assistance.

Problem 4: NTE raises errors or behaves unexpectedly on large topologies (>100K nodes)

Cause: NTE has been benchmarked against 100K-device synthetic topologies. Beyond this scale, the dual-write architecture (petgraph + DataFrameStore) may exhibit performance degradation or, in rare cases, desynchronisation. The dual-write has been hardened with rollback safety but is not formally verified.

Solution: For topologies approaching 100K nodes, switch NTE to the Lite datastore backend (`nte-datastore-lite`) which avoids the dual-write path. Check the NTE release notes for any scale advisories. File a bug report with your topology size and the operation that triggered the issue.

Problem 5: deviceinteraction CLI not available / device tests cannot be run

Cause: The deviceinteraction user-facing CLI (`di` command) is in planning (Phase 8) and has not shipped. The core crate (`di-core`) and verification framework are implemented, but they are not yet exposed as an end-user tool.

Solution: Device validation can be approximated using autonetkit-analysis with an `.operational.json` snapshot produced by configparsing (if you have vendor configs) or by piping raw CLI output through the configparsing extraction pipeline. Monitor the deviceinteraction repository for CLI releases.

Problem 6: netassure subcommands not found or produce no output

Cause: netassure v0.1 is in architecture phase only. All five analysis paradigms (`verify`, `analyze`, `cascade`, `predict`, `optimize`) are specified but not yet implemented. The package installs successfully but most commands return stubs.

Solution: Do not include netassure in production or evaluation pipelines until v0.2 (targeted for Q2 2026, Phase 14). For formal verification needs, use an external tool such as Batfish. For graph-based analysis, query NTE directly via autonetkit's Python API.

Problem 7: topogen produces different IDs on re-run despite same seed

Cause: ID stability requires that the same seed, template, and node count produce identical IDs. ID instability is most commonly caused by a version mismatch between topogen invocations (different minor versions may change the determinism algorithm).

Solution: Pin topogen to a specific version in your pipeline: `cargo install topogen --version 1.5.x`. Run `topogen show <file> --ids` to list all IDs and compare across runs. If IDs still differ, check that no environment variables (e.g., `TOPOGEN_SEED`) are overriding the `--seed` flag.

Problem 8: ank validate passes but ank-netcfg compile fails

Cause: The autonetkit validator and ank-netcfg compiler have independent rule sets at this stage of development. autonetkit checks design correctness (31 rules); ank-netcfg checks compilability for a specific vendor platform. A design can be structurally valid but use a protocol feature not yet supported by a compiler target.

Solution: Run `ank-netcfg check <design.yaml> --platform <target>` to pre-validate compilability before the full compile run. Consult NETCFG-UM-001 for the supported feature matrix per platform.

8 Cross-References to Individual Manuals

Each tool has a dedicated user manual. This ecosystem manual covers cross-tool integration; tool-internal depth (full CLI reference, configuration schema, algorithms, benchmarks) is in the per-tool manuals.

8.1 Generation & Modelling

See also: *TopoGen User Manual* (TOPOGEN-UM-001) — All nine topology generator templates, traffic matrix configuration, overlay tagging (BGP, OSPF, Physical), SRLG fate-sharing, and export format details.

See also: *AutoNetKit User Manual* (ANK-UM-001) — Type-safe network modeling, the 31 design validation rules, query API, IP allocation, and the `autonetkit-config` compiler.

See also: *NTE User Manual* (NTE-UM-001) — Graph-relational query engine, datastore backends, event sourcing, Cypher-inspired query syntax, and scale benchmarks.

See also: *NetCfg User Manual* (NETCFG-UM-001) — ank-netcfg compilation pipeline, vendor platform coverage, MiniJinja template customization, and trace log format.

8.2 Simulation & Analysis

See also: *netsim User Manual* (NETSIM-UM-001) — Protocol simulation (OSPF, BGP, IS-IS), failure scenario configuration, convergence auditing, daemon mode, and the gRPC interactive terminal API.

See also: *Spectral/netflowsim User Manual* (SPECTRA-UM-001) — Flow-based traffic analysis, queuing models (M/D/1, M/M/1/K), Monte Carlo configuration, failure scenario syntax, and FIB bridge details.

See also: *Passive/netassure User Manual* (PR-UM-001) — The five analysis paradigms (formal verification, graph algorithms, failure cascade, ML/GNN, optimization), telemetry ingestion pipeline, GNN training workflow, and MLflow model registry. Note: v0.1 is architecture-only; implementation targeted Q2 2026.

8.3 Visualization & Orchestration

See also: *netvis User Manual* (NETVIS-UM-001) — Layout algorithms (force-directed, hierarchical, Barnes-Hut), overlay types, WASM rendering, deterministic layout mode, and SVG/HTML/PNG export.

See also: *Workbench User Manual* (ANK-WB-UM-001) — Web IDE project management, YAML editor configuration, simulation panel, interactive terminal, FIB bridge traffic analysis, and RFC-02 project manifest management.

8.4 Device Interaction & Config Parsing

See also: *DeviceInteraction User Manual* (PR-UM-001, shared volume) — Testbed YAML format, SSH/Telnet/REST connection pooling, trigger and verification framework, and data-driven test definitions. Note: CLI tool not yet shipped (Phase 8 planned).

9 Further Reading

- *Network Automation Ecosystem Technical Report* (NETAUTO-TR-001) — Architecture design rationale, the full data contract specifications (RFC-01, RFC-02, RFC-03), integration mechanism details, Architecture Decision Records (ADR-0001 through ADR-0005), competitive positioning, and the implementation roadmap. Read this document for the *why* behind every design choice described in this manual.
- *RFC-01: Canonical Data Contract v2.0* (`RFC-01.md`) — The normative specification for the multi-file sidecar strategy, ID conventions, and per-tool obligations.
- *RFC-02: Project Manifest* (`RFC-02.md`) — The normative specification for `netauto.project`, provider types, artifact dependency tracking, and the Live Overlay Stream v1.0 WebSocket contract.
- *Ecosystem Interoperability Strategy* (`ECOSYSTEM_INTEROP.md`) — A practitioner guide to multi-source topology ingestion, identity resolution across heterogeneous sources (NetBox, CLI scrape, cloud APIs), and provenance tracking.